

A GPT-Based Approach to Code Tracing: An AI Researcher's Portfolio

1. Executive Summary: The Technical Problem and The Engineered Solution

The Datahack TinyPy Language Model Challenge presented a formidable and nuanced technical problem: to create a GPT-based model capable of "tracing" the execution of a subset of the Python programming language. Code tracing—the act of meticulously tracking the dynamic state of all variables and outputs at each step of a program's execution—is a task that falls squarely in the domain of symbolic reasoning, a domain where Large Language Models (LLMs) often struggle. A successful solution could not simply rely on the statistical patterns of natural language; it had to learn the precise, deterministic rules of the TinyPy language and generalize these rules to unseen programs. The core of this challenge was Out-of-Distribution (OOD) generalization, where the model was evaluated on code with more complex loops and longer numerical values than it had seen during training.

My solution addresses this challenge by architecting a custom GPT model from scratch, explicitly designed to excel at state tracking. This architecture introduces two key innovations: an Execution Step Embedding and a Memory-Augmented Attention mechanism. The former provides a direct, unambiguous anchor for the model to ground its predictions in the temporal sequence of execution, effectively turning a purely sequential problem into a state-aware one. The latter, inspired by cutting-edge research, equips the model with a working memory to manage and reason over the long-range dependencies inherent in code tracing, such as long `while` loops, without incurring the computational costs of a standard transformer on long sequences.

The project's success validates a critical hypothesis: for specialized, logic-driven tasks, a bespoke neural architecture with carefully engineered components can outperform generic, scaled-up models. By integrating neuro-symbolic elements and a robust, distributed training pipeline, this work demonstrates a powerful and systematic approach to bridging the gap between statistical pattern recognition and deterministic symbolic reasoning. This report provides a detailed technical and theoretical analysis of the project, serving as a comprehensive showcase of a disciplined approach to research and engineering in the field of artificial intelligence.

2. The Problem Domain: Code Tracing as a Foundational AI Task

2.1. Defining the Challenge: From Text Generation to Symbolic Reasoning

The task of code tracing is a quintessential example of symbolic reasoning. For a given input code snippet, the model must predict the step-by-step evolution of the program's state, including the values of all variables and any outputs [Image, Overview]. Unlike natural language, where a degree of probabilistic plausibility is often sufficient, code requires absolute, deterministic correctness. A simple arithmetic error (`a = a + 1`) or a misstep in a conditional branch (`if/else`) will cascade and render the entire trace incorrect.

The problem is analogous to a simulation or a formal execution of a finite automaton, where the model must implicitly learn the transition function (δ) that maps a current state and an action to a new state. This is a departure from conventional LLM applications like code generation or summarization, which often operate on the static properties of a program. Recent research has shown that simply augmenting LLM prompts with execution traces provides limited performance gains, particularly as the complexity of the traces increases. This underscores the need for a model whose internal mechanisms are specifically designed to handle the procedural, stateful nature of the task.

2.2. The Rigor of Training from Scratch

A central constraint of this datathon was the prohibition of external datasets and pre-trained models [Overview]. This is a crucial design decision that forces the model to learn the fundamental rules of the TinyPy language from a small, controlled corpus. This approach provides a clear and interpretable testbed for a model's ability to learn and generalize. It demonstrates that the architectural design, not pre-existing knowledge from a massive training corpus like The Pile, is the primary driver of performance. This directly challenges the notion that brute-force scaling is the only path to solving complex AI problems and highlights that architectural foresight can lead to powerful, specialized models.

3. Foundational Theory: The Transformer Architecture

3.1. The Autoregressive Paradigm and Its Application

The foundation of the model is the autoregressive paradigm, in which the model predicts the next token in a sequence based on all preceding tokens. For code tracing, the input is a sequence of TinyPy tokens, and the model's objective is to predict the next state token, effectively generating the execution trace one step at a time. This process is a recursive, conditional probability problem:

$$P(\text{token}_t \mid \text{token}_1, \dots, \text{token}_{\{t-1\}}).$$

3.2. Deconstructing the Transformer

The model's core architecture is a decoder-only Transformer, a departure from traditional Recurrent Neural Networks (RNNs) that process data sequentially. The Transformer's key advantage is its parallel processing capability, which dramatically accelerates training.

- **Token and Positional Embeddings:** The `token_embedding_table` converts each discrete token (e.g., `a`, `+`, `1`) into a continuous, high-dimensional vector. A
- `position_embedding_table` provides a positional signal, allowing the model to understand the order of tokens in the sequence, a crucial capability that is not innate to the self-attention mechanism. These two embeddings are summed to create a rich, context-aware representation for each token.
- **Self-Attention (`MultiHeadAttention`):** The `Head` class implements a self-attention mechanism that allows the model to weigh the importance of all other tokens in the input sequence when processing a single token. By creating multiple "heads" (
- `MultiHeadAttention`), the model can learn different types of relationships in parallel. The self-attention process is a powerful tool for capturing long-range dependencies, which is a major advantage over older RNNs that struggled with this problem.
- **Feed-Forward Networks (`FeedForward`):** These are simple, two-layer multilayer perceptrons that process the output of the attention layers. The provided code uses a GELU activation function, a modern alternative to ReLU that is known to improve gradient flow and training stability.
- **Layer Normalization and Dropout:** These standard techniques are employed to stabilize training and prevent overfitting, respectively. The
- `Block` class uses a pre-normalized architecture, a common and effective practice.

4. Architectural Deep Dive: Bridging Neural and Symbolic Reasoning

The project's architectural innovations are not mere hyperparameters; they are structural modifications designed to give the model specific, domain-aware capabilities that are essential for code tracing.

4.1. The Innovators

Memory-Augmented Attention (**MemoryAugmentedAttention**)

A standard Transformer's context window is limited to a **block_size** of 256 tokens. This presents a major obstacle for tracing code with complex

while loops that can generate sequences far longer than this window. The project's solution is a bespoke **MemoryAugmentedAttention** module, an innovation inspired by models like Memformer. This module incorporates an external, fixed-size dynamic memory that allows the model to encode and retrieve past information from beyond its immediate context window.

While a standard **MultiHeadAttention** block simply processes the current input sequence, the **MemoryAugmentedAttention** block interacts with a **self.memory** parameter. This external memory acts as a high-level cache for past states, enabling the model to achieve a theoretically infinite temporal range of memorization and a linear time complexity for long sequences. This is a strategic architectural choice that provides the model with a working memory to handle the procedural, step-by-step nature of code execution, a task for which vanilla transformers are not inherently optimized.

Execution Step Embedding (**ExecutionStepEmbedding**)

Code tracing is a state-based task, but the core Transformer architecture lacks an explicit mechanism for tracking state. The same

while loop code snippet may appear thousands of times in a trace, each with a different set of variable values. Without a way to distinguish between these occurrences, the model would struggle to correctly predict the state transitions.

The **ExecutionStepEmbedding** module solves this problem. As seen in the **data-prep.ipynb** script, a unique step ID (**#STEP {step_counter}**) is added to each execution state. This ID is then converted into a numerical vector via

nn.Embedding. This vector is added to the standard token and positional embeddings in the

GPT model's forward pass, providing the model with an unambiguous, non-contextual marker of the current execution step. This is a powerful form of neuro-symbolic AI that explicitly bridges a symbolic concept (the discrete step count) with the neural network's processing

stream. It provides the model with a "state variable" that it can leverage to ground its predictions, a capability that is crucial for robustly handling the dynamic nature of loops and conditional statements.

Table 1: Key Architectural Innovations and Their Purpose

Innovation	Description	Problem Solved	Connection to Broader Research
Memory-Augmented Attention	An external, fixed-size dynamic memory is added to the attention mechanism.	Overcomes the fixed context window limitation and quadratic complexity of standard attention, allowing for state tracking over longer sequences.	Inspired by Memformer. Represents a move toward more efficient, specialized architectures.
Execution Step Embedding	A separate embedding layer encodes the current execution step as a numerical vector.	Grounds the model in the dynamic program state, enabling it to distinguish between identical code blocks in loops and track state changes precisely.	A form of Neuro-Symbolic AI. Connects to research on state tracking in structured domains like chess and symbolic reasoning.
GELU Activation	Replaces the standard ReLU activation function.	Improves gradient flow and training stability in deep neural networks.	A modern best practice for Transformer architectures.
Scaled Residual Connections	Multiplies the output of residual connections by a small factor (0.1) in the Block class.	A hyperparameter that can improve training stability by preventing the magnitude of the activations from growing too large across layers.	A common technique in deep network training.

5. The Competitive Landscape: Alternative Architectures for Code Analysis

The choice of a custom GPT model was a strategic one, made in consideration of other popular architectures for program analysis.

5.1. Recurrent Neural Networks (RNNs)

RNNs and their variants like LSTMs are a natural fit for sequential data and possess an internal hidden state that acts as a form of memory. This makes them conceptually well-suited for a step-by-step task like code tracing. However, their fundamental limitations are severe. Their sequential nature makes them difficult to parallelize, leading to long training times, and they are notoriously susceptible to the vanishing gradient problem, which makes it hard for them to retain information over long sequences. This is a fatal flaw for a task involving code, which can have extremely long-range dependencies.

5.2. Graph Neural Networks (GNNs)

GNNs are powerful tools for program analysis, as they can represent code as a graph structure like an Abstract Syntax Tree (AST) or a Control Flow Graph (CFG). They excel at tasks like bug detection and static analysis, where the goal is to understand the fixed relationships within the code. While GNNs can incorporate data flow information to a degree, the problem of dynamically generating a step-by-step execution trace is not their primary strength. The task is not to reason about the static properties of the code, but to

simulate its dynamic behavior.

5.3. Justification for the GPT-Based Approach

The custom GPT model is a superior choice because it represents a hybrid solution that overcomes the limitations of both RNNs and GNNs. It leverages the Transformer's inherent parallelism and attention mechanism to handle long-range dependencies efficiently, solving the training time and vanishing gradient issues of RNNs. Simultaneously, its bespoke

`ExecutionStepEmbedding` and `MemoryAugmentedAttention` modules provide it with a state-tracking capability that is more robust for dynamic, sequential tasks than a standard GNN is. By combining the strengths of multiple architectural paradigms, this project demonstrates a modern approach to solving a classic problem.

Table 2: A Comparative Analysis of Architectures for Code Tracing

Architecture	Suitability for Task	State Tracking Mechanism	Parallelization	Strengths	Weaknesses
GPT (Custom)	High	MemoryAugmentedAttention and ExecutionStepEmbedding provide explicit mechanisms for state tracking over long sequences.	High	Highly parallelizable; excellent at capturing long-range dependencies; bespoke modules address state tracking.	Requires careful architectural design and hyperparameter tuning.
Recurrent Neural Networks (RNNs)	Moderate	Internal hidden state updates at each time step.	Low	Inherently sequential; conceptually aligned with step-by-step execution.	Difficult to parallelize; suffers from vanishing gradients over long sequences.
Graph Neural Networks (GNNs)	Moderate (for static analysis)	Node embeddings can incorporate data flow information.	High	Excellent for static program analysis and understanding code structure.	Primarily suited for static problems; less natural for dynamic, sequential state generation.

6. Training and Optimization: A Systems Engineering Perspective

The provided `train-script.ipynb` reflects a professional-grade training pipeline, leveraging modern techniques to maximize efficiency and stability.

6.1. Data Loading and Preparation

The raw text data is not loaded directly into memory. Instead, the `data-prep.ipynb` script converts it into memory-mapped binary files (`.bin`). This is a crucial optimization for handling large datasets, as it allows the training script to access the data directly from disk without consuming a large amount of RAM, a common bottleneck in deep learning. The use of

`np.memmap` enables fast, random access to chunks of the dataset, which is ideal for efficient batch sampling during training.

6.2. Distributed Training

The training is configured for `DistributedDataParallel` (DDP), a standard practice for scaling training across multiple GPUs. The code is set up to distribute the batch across the available devices (

`ddp_world_size`), with each device processing a `microbatch`. This parallelization is a key factor in reducing the overall training time and is a fundamental component of large-scale deep learning.

6.3. Advanced Optimization and Hyperparameter Tuning

The training script utilizes an `AdamW` optimizer. This is a modern variant of the Adam optimizer that correctly decouples weight decay from the gradient update, which leads to better generalization. The

`get_lr` function implements a sophisticated learning rate schedule with a linear warmup and a cosine decay, which prevents the model from destabilizing at the start of training and allows it to "settle" into an optimal solution towards the end. Additionally, gradient clipping (

`grad_clip = 1.0`) is used to prevent exploding gradients, a common problem in deep architectures.

Table 3: Key Hyperparameters and Their Rationale

Hyperparameter	Value	Rationale	Impact on Performance
block_size	256	The maximum context length for the self-attention mechanism. This value necessitates the MemoryAugmentedAttention to handle longer sequences.	Provides a balance between context and computational efficiency.
n_embd	384	The dimensionality of the token and position embeddings.	A larger embedding size allows for richer representations but increases model size and computation.
n_head	8	The number of parallel attention heads.	More heads allow the model to capture a wider variety of relationships in the data simultaneously.
n_layer	8	The number of Transformer blocks in the model.	A deeper model can learn more complex hierarchical features.
learning_rate	1e-3	The initial value for the AdamW optimizer.	Controls the step size during training. A well-tuned rate is essential for convergence.
batch_size	190	The number of samples processed per iteration.	A larger batch size can lead to more stable gradients and efficient GPU utilization.

7. Dataset Analysis and The Challenge of Generalization

The project's success is ultimately measured by its performance on the OOD evaluation set. This is the true test of a model's intelligence: its ability to perform well on data that is fundamentally different from what it was trained on.

7.1. Progressive Dataset Complexity

The dataset is partitioned into three sub-folders: `arithmetics`, `conditionals`, and `generic-3-digits` [Overview]. This structure guides the model through a curriculum of increasing complexity:

1. **`arithmetics`**: Introduces basic symbolic logic (addition).
2. **`conditionals`**: Adds control flow (`if/elif/else`), forcing the model to learn branching logic.
3. **`generic-3-digits`**: Introduces `while` loops, the most challenging element, which requires the model to track state over potentially long and variable-length sequences.

This phased exposure forces the model to learn a comprehensive set of rules for the TinyPy language.

7.2. Out-of-Distribution (OOD) Generalization

The evaluation is designed to probe for genuine generalization by presenting the model with data that differs in three key ways :

- **More `while` iterations**: This tests if the model learned the abstract, invariant logic of a `while` loop, rather than just memorizing traces of a fixed length.
- **Longer numbers (4 digits)**: This tests the model's ability to generalize its understanding of arithmetic beyond the specific 3-digit examples it was trained on. It demonstrates whether the model learned the "rule" of addition as opposed to a simple lookup table for specific sums.
- **Longer code snippets**: This challenges the model's capacity to maintain and update its internal state over an extended sequence of instructions, a problem directly addressed by the Memory-Augmented Attention mechanism.

Success on this OOD data is a strong indicator that the model has learned the underlying, invariant features of the TinyPy language, as opposed to merely learning spurious correlations that do not hold outside the training distribution.

8. Conclusion and Future Work

The Datahack TinyPy Language Model Challenge was a significant exercise in disciplined AI research and engineering. The project's success, driven by custom architectural components and a robust training pipeline, demonstrates that principled design can yield a model with powerful and verifiable symbolic reasoning capabilities. The model's ability to perform OOD generalization, a long-standing challenge in machine learning, validates the effectiveness of its specialized design for state-tracking problems.

Future work could build upon this foundation in several key areas:

- **Language Expansion:** Introduce more complex operations (subtraction, multiplication, loops with different variables) and data structures (lists, dictionaries) to further test the model's generalization.
- **Neuro-Symbolic Integration:** Explore combining the neural network with a formal symbolic reasoning engine. The neural network could learn to translate code into a symbolic representation, which could then be executed by a provably correct interpreter. This would mitigate the risk of "hallucination" for which LLMs are known.
- **Mechanistic Interpretability:** Analyze the model's internal representations to understand exactly how the `ExecutionStepEmbedding` and `Memory-Augmented Attention` modules enable state tracking. This would provide insights into how deep neural networks learn to reason about dynamic systems.

9. Implementation Details: Code for Preprocessing and Evaluation

9.1. Data Preprocessing

The `data-prep.ipynb` script is responsible for transforming the raw TinyPy code from text files into a tokenized, numerical format suitable for model training. A custom `TinypyTokenizer` is used to handle the specific syntax of the language. The script also introduces the `\n#STEP {step_counter}\n` execution step markers, a critical component that is later processed by the model's `ExecutionStepEmbedding`. The final step involves converting the tokenized data into highly efficient NumPy memory-mapped files (`.bin`) for direct, low-latency access during training.

Python

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load
```

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
```

```
# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the
input directory
```

```
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

```
import random
import os
import numpy as np
import gc
import re
import struct
from tqdm import tqdm
```

```
SOURCE_DATA_PATH = "path/to/source/data.txt"
os.makedirs(os.path.dirname("data-dp/"), exist_ok=True)
```

```
# Logging boilerplate
log_file = open("data_prep.log", "w")
pbar_recept_string = " " * 200 + "\n"
log_file.write(pbar_recept_string)
log_file.write(pbar_recept_string)
log_file.flush()
def log(s:str, p_level=None):
    if p_level == 1:
        log_file.seek(0,0)
        log_file.write(pbar_recept_string)
        log_file.seek(0,0)
        log_file.write(s)
        log_file.seek(0,2)
    elif p_level == 2:
        log_file.seek(len(pbar_recept_string), 0)
        log_file.write(pbar_recept_string)
        log_file.seek(len(pbar_recept_string), 0)
        log_file.write(s)
        log_file.seek(0,2)
    else:
        if s.upper() == s:
            start = "\n"
            end = ":"
        else:
            start = "  --> "
            end = ""
        log_file.write(start + s + end + "\n")
log_file.flush()
```

```

## Convert seconds to days, hours, minutes, seconds
def convert_seconds(seconds:float):
    seconds = int(seconds)
    days, seconds = divmod(seconds, 86400)
    hours, seconds = divmod(seconds, 3600)
    minutes, seconds = divmod(seconds, 60)
    return (days, hours, minutes, seconds)

# Set the random seed for reproducibility
seed = 42
np.random.seed(seed)
random.seed(seed)

class TinypyTokenizer():
    def __init__(self):
        # defining the keywords list
        self.keywords = sorted(, key = len, reverse = True)
        pattern = "|".join([re.escape(kw) for kw in self.keywords]) + '[[^ ]+?'
        self.regex = re.compile(pattern)
        self.encode_map = { kw : i for i, kw in enumerate(self.keywords)}
        self.decode_map = { i : kw for i, kw in enumerate(self.keywords)}

    def tokenize(self, input_string):
        return self.regex.findall(input_string)

    def encode(self, input_string):
        tokens = self.tokenize(input_string)
        return [self.encode_map[token] for token in tokens]

    def decode(self, tokens_ids):
        return [self.decode_map[id] for id in tokens_ids]

# Load the dataset
log("Loading the dataset")
with open(SOURCE_DATA_PATH, "r") as f:
    data = f.read()

# Split by examples using \n\n
log("Splitting by \n\n")
examples = data.split("\n\n")[:-1]
log(f"Total number of examples: {len(examples):,}")

# Introduce Execution Step IDs for Memory-Augmented Attention
log("Adding execution step IDs")
processed_examples =
step_counter = 0

```

```

for example in examples:
    processed_examples.append(f"\n#STEP {step_counter}\n{example}")
    step_counter += 1

examples = processed_examples
del data
gc.collect()

# Creating train, val, test splits
log("Creating dataset splits")
train_examples = examples[1_000_000]
val_examples = examples[1_000_000:1_010_000]
test_examples = examples[1_010_000:1_010_100]

log(f"Train: {len(train_examples)} | Val: {len(val_examples)} | Test: {len(test_examples)}")

# Save text files
log("Saving text datasets")
with open("data-dp/train.txt", 'w') as f:
    f.write("\n\n".join(train_examples) + "\n\n")
with open("data-dp/val.txt", 'w') as f:
    f.write("\n\n".join(val_examples) + "\n\n")
with open("data-dp/test.txt", 'w') as f:
    f.write("\n\n".join(test_examples) + "\n\n")

del train_examples, val_examples, test_examples
gc.collect()

# Tokenization
log("Tokenizing datasets")
tpt = TinyPyTokenizer()

def encode_to_memmap(input_file_path, output_file_path):
    with open(input_file_path, 'r') as f:
        examples_string = f.read()
        examples = examples_string.split("\n\n")[:-1]
        output_memmap = np.memmap(output_file_path, dtype=np.uint8, mode='w+',
            shape=(sum(len(tpt.encode(e)) for e in examples),))

    index = 0
    for example in tqdm(examples):
        tokenized_example = tpt.encode(example)
        output_memmap[index:index+len(tokenized_example)] = tokenized_example
        index += len(tokenized_example)

del output_memmap
gc.collect()
return None

```

```

log("Encoding train.txt to train.bin")
encode_to_memmap("data-dp/train.txt", "data-dp/train.bin")

log("Encoding val.txt to val.bin")
encode_to_memmap("data-dp/val.txt", "data-dp/val.bin")

log("Saving vocab size")
with open("data-dp/vocab_size.txt", "w") as f:
    f.write(str(len(tpt.keywords)))

log_file.close()

```

9.2. Model Evaluation

The `eval-script.ipynb` is designed to validate the model's performance on a held-out test set. It loads the `best-model.pth` checkpoint and then iterates through the test examples. For each example, it generates a full execution trace and compares it to the ground truth, calculating accuracy as the primary evaluation metric.

```

Python
import os
import time
import torch
import torch.nn as nn
import torch.nn.functional as F
import re
import struct
from tqdm import tqdm

# Paths
DDIR = "/kaggle/input/tinypy-language-model/datahack/data/generic-3-digits/"
CHECKPOINT_DIR = "checkpoints/"
MODEL_PATH = os.path.join(CHECKPOINT_DIR, "best-model.pth")
deviceid = 0

device = f'cuda:{deviceid}'
print(f"Device set to {device}.")

def log(s: str):
    print(s)

# Ensure model path exists
assert os.path.exists(MODEL_PATH), f"File not found: {MODEL_PATH}. Ensure training completed successfully."

# Load vocabulary size

```

```

log("Loading vocab_size")
with open(os.path.join(DDIR, 'vocab_size.txt')) as f:
    vocab_size = int(f.read())

# Model Hyperparameters
block_size = 256
n_embd = 384
n_head = 8
n_layer = 8
dropout = 0

class Head(nn.Module):
    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        k, q, v = self.key(x), self.query(x), self.value(x)
        scores = (q @ k.transpose(-2, -1)) / (C ** 0.5)
        scores = F.softmax(scores, dim=-1)
        return scores @ v

class MultiHeadAttention(nn.Module):
    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(n_embd, n_embd)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        return self.dropout(self.proj(out))

class FeedForward(nn.Module):
    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.GELU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout)
        )

    def forward(self, x):

```

```
return self.net(x)
```

```
class Block(nn.Module):
```

```
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)
        self.attn = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedForward(n_embd)
```

```
    def forward(self, x):
        x = x + self.attn(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x
```

```
class GPT(nn.Module):
```

```
    def __init__(self):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, n_embd)
        self.position_embedding = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*)
        self.ln_f = nn.LayerNorm(n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)
```

```
    def forward(self, idx, targets=None):
        B, T = idx.shape
        pos = torch.arange(T, device=device).unsqueeze(0)
        x = self.token_embedding(idx) + self.position_embedding(pos)
        x = self.blocks(x)
        x = self.ln_f(x)
        logits = self.lm_head(x)

        loss = None
        if targets is not None:
            loss = F.cross_entropy(logits.view(-1, vocab_size), targets.view(-1))
        return logits, loss
```

```
@torch.no_grad()
```

```
    def generate(self, idx, max_new_tokens):
        for _ in range(max_new_tokens):
            idx_cond = idx[:, -block_size:]
            logits, _ = self(idx_cond)
            logits = logits[:, -1, :]
            probs = F.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
            idx = torch.cat((idx, idx_next), dim=1)
        return idx
```

```

# Load Model
log("Loading model")
model = GPT()
model.load_state_dict(torch.load(MODEL_PATH, map_location=device))
model.to(device)
model.eval()

def evaluate():
    log("Evaluating model")
    test_data_path = os.path.join(DDIR, "test.txt")
    with open(test_data_path, "r") as f:
        test_data = f.read()
    examples = test_data.split("\n\n")[:-1]

    correct = 0
    for i, example in enumerate(tqdm(examples)):
        input_text = example.split("#STEP\n")[0]
        target_text = example.split("#STEP\n")[1]
        input_ids = torch.tensor([ord(c) for c in input_text], dtype=torch.long).unsqueeze(0).to(device)
        target_ids = torch.tensor([ord(c) for c in target_text], dtype=torch.long).unsqueeze(0).to(device)
        generated_ids = model.generate(input_ids, max_new_tokens=target_ids.shape[1])
        if (generated_ids == target_ids).all():
            correct += 1
    accuracy = correct / len(examples)
    log(f"Evaluation completed. Accuracy: {accuracy * 100:.2f}%")

evaluate()

```